

On-Board Real Time 3D LiDAR-IMU SLAM for UAV Subterranean Mapping

Stepan Letsko[†], Engin Zeydan^{*}, Madhusanka Liyanage[‡], and Pasika Ranaweera[†]

[†]School of Electrical and Electronic Engineering, University College Dublin, Ireland

^{*}Centre Tecnològic de Telecomunicacions de Catalunya (CTTC/CERCA), Castelldefels, Spain

[‡]Network Softwarization and Security Labs (NetsLab), School of Computer Science, University College Dublin, Ireland

Email:stepan.letsko@ucdconnect.ie, engin.zeydan@cttc.cat, madhusanka@ucd.ie, pasika.ranaweera@ucd.ie

Abstract—Subterranean mapping is essential for geological research and search-and-rescue operations. Traditional manual surveying tools are ineffective in hazardous, GPS-denied, and unlit environments, making it necessary to use autonomous unmanned aerial vehicles (UAVs) equipped with LiDAR-inertial sensors. However, state-of-the-art LiDAR-inertial odometry algorithms often exceed the size, weight, and power constraints of lightweight aerial platforms, resulting in memory saturation and processing bottlenecks. This paper proposes and implements an edge-based split-compute architecture for real-time mapping onboard a resource-constrained UAV. The system uses the FAST-LIO2 algorithm for edge-based survival odometry and offloads global mapping to a remote base station. The paper evaluates key mapping accuracy and computational performance metrics, including Absolute Pose Error (APE), CPU usage, and RAM utilization onboard the Raspberry Pi 5. Additionally, it quantitatively analyzes optimal parameters for onboard processing across a 25-configuration parameter sweep, identifying a voxel downsampling resolution that maintains a safe 55.1 ms processing latency with zero frame drops. The proposed architecture is validated through physical deployment across three geometrically diverse environments, achieving sub-metre closing error in every case, including a 0.53 m closing error under near-darkness, GPS-denied conditions directly representative of the target subterranean scenario. Baseline accuracy was further validated on the NCLT benchmark, achieving a Root Mean Square Error (RMSE) of 0.908 m (0.07% mean translational error) over a 1.146 km trajectory.

I. INTRODUCTION

Subterranean environments, including natural cave networks and artificial mining tunnels, are valuable for geological research and are often the sites of complex search-and-rescue (SAR) operations. Traditional manual surveying of these environments is very time-consuming and inherently dangerous due to hazards such as unstable terrain and rockfalls. The urgent need for rapid, accurate 3D mapping is underscored by incidents such as the 2018 Tham Luang cave rescue in Thailand, where a trapped football team and rescue operators had to rely on incomplete, decades-old 2D schematics, which severely hindered the extraction timeline [1]. Deploying autonomous Unmanned Aerial Vehicles (UAVs) provides an effective solution for rapid spatial data acquisition without endangering human lives. However, operating autonomous UAVs in subterranean systems presents significant navigational and computational challenges. Deep caves are strictly GPS-denied environments and completely lack natural illumination,

making standard Global Navigation Satellite Systems (GNSS) and Visual Simultaneous Localization and Mapping (vSLAM) frameworks ineffective [2]. Although 3D mechanical Light Detection and Ranging (LiDAR) fused with high-frequency Inertial Measurement Units (IMUs) provides robust, high-fidelity perception in absolute darkness [3], the resulting state estimation algorithms are highly computationally intensive. To navigate, the UAV must perform real-time SLAM to estimate its trajectory while simultaneously processing hundreds of thousands of laser points per second.

Processing these dense 3D point clouds on a highly mobile platform introduces severe computational bottlenecks. State-of-the-art LiDAR-inertial odometry algorithms are typically designed for ground vehicles or powerful desktop computers. When ported to lightweight UAVs, these algorithms frequently exceed the strict Size, Weight, and Power (SWaP) capacities of embedded processors. This mismatch leads to CPU core saturation, memory overflow, and ultimately, catastrophic navigation failure during flight. To address these hardware limitations, this paper investigates the operational boundaries of embedded SLAM and proposes a highly optimised software architecture tailored for resource-constrained UAVs. The specific contributions of this work are as follows.

- The design and implementation of an edge-cloud split-compute architecture optimised for the ARM-based Raspberry Pi 5 using ROS 2. This architecture isolates mission-critical survival odometry at the edge while offloading resource-intensive global mapping to a remote base station.
- A rigorous quantitative analysis of embedded SLAM limitations, specifically identifying the RAM and CPU saturation thresholds that cause system failure during global map generation.
- The empirical determination of optimal operational parameters, such as voxel grid spatial resolutions, required to prevent core saturation while maintaining sufficient geometric constraints for accurate state estimation on ARM-based processors.

II. RELATED WORK

The autonomous exploration of subterranean environments has historically presented significant navigational challenges due to complex terrain, confined spaces, and the complete absence of GPS and ambient light. To address these hazards,

research has increasingly shifted from teleoperated rovers to fully autonomous, multi-modal robotic deployments. Early aerial systems attempted to overcome the lack of light using active visual sensing. For example, Zhang et al. [4] developed SmartCaveDrone, which used RGB-D sensors paired with active lighting. However, active illumination significantly increases power consumption and payload weight, severely limiting the flight endurance of micro-UAVs. Similarly, Papachristos et al. [2] evaluated Near-Infrared (NIR) stereo cameras combined with Time-of-Flight (ToF) sensors. While these visual methods reduced reliance on visible light, they still showed poor performance and rapid tracking failure in feature-sparse, homogeneous rock environments.

Consequently, 3D mechanical LiDAR has become the primary perceptual sensor for subterranean environments, providing high-fidelity spatial data without the need for external illumination. Early LiDAR applications by Weishampel et al. in [5] were limited to airborne platforms identifying cave entrances through forest canopies, but recent advancements have focused on internal volumetric mapping. For example, Grasso et al. in [6] mapped the Bossea Cave using LiDAR-based SLAM technology. Although the system provided raw odometry, achieving a globally consistent map required more than three hours of offline post-processing for loop closures and trajectory optimisation. This heavy reliance on post-flight computation makes the approach unsuitable for active UAV deployment, which requires simultaneous, real-time mapping and state estimation to enable autonomous navigation.

The push for real-time, autonomous aerial mapping has been greatly accelerated by the DARPA Subterranean (SubT) Challenge. Winning deployments, such as the CERBERUS system-of-systems presented by Tranzatto et al. in [7], successfully demonstrated UAVs performing LiDAR-inertial SLAM in complex underground networks. However, to manage the computational demands of the SLAM and artifact detection algorithms, these state-of-the-art UAVs required large, high-power payloads, specifically, a 16-thread AMD 4800U processor, 64 GB of RAM, and three Intel Neural Compute Sticks. This significant Size, Weight, and Power (SWaP) penalty severely limited their flight endurance. A comparison of these different cave exploration modalities and their limitations is summarised in Table I.

Processing dense LiDAR point clouds directly on lightweight micro-UAVs creates significant software bottlenecks. Early algorithms such as LOAM [8] have difficulty with spatial degeneracy in unstructured, non-planar cave geometries. Tightly coupled factor-graph frameworks like LIO-SAM [9] integrate IMU data and global loop closures to address drift; however, maintaining a keyframe-based graph quickly exceeds the RAM and CPU capacities of embedded processors, making them impractical for extended autonomous flights. To address strict SWaP constraints, algorithms such as FAST-LIO2 [10] use direct methods and incremental kd-trees to achieve highly efficient local odometry natively on ARM processors. However, no previous work has successfully implemented high-frequency LiDAR-inertial odometry with global

loop closures entirely within the hardware limits of a micro-UAV payload in real time. This research addresses this gap by proposing an edge-cloud split architecture optimised for the ARM Cortex-A76 cores of a Raspberry Pi 5. By running survival odometry on the embedded edge and offloading global pose-graph optimisation, this approach overcomes the hardware bottlenecks that severely limit autonomous subterranean platforms.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

Fig. 1 presents the conceptual overview of the proposed edge-assisted UAV subterranean mapping framework, illustrating the onboard sensing and odometry functions, wireless telemetry link, and cloud-assisted global mapping pipeline. This section describes the hardware and software framework developed to perform LiDAR-inertial odometry and mapping on a resource-constrained UAV. It outlines the physical sensor payload, the mathematical formulation of the state estimator, the telemetry pipeline, and the proposed edge-cloud split architecture designed to address embedded processing limitations.

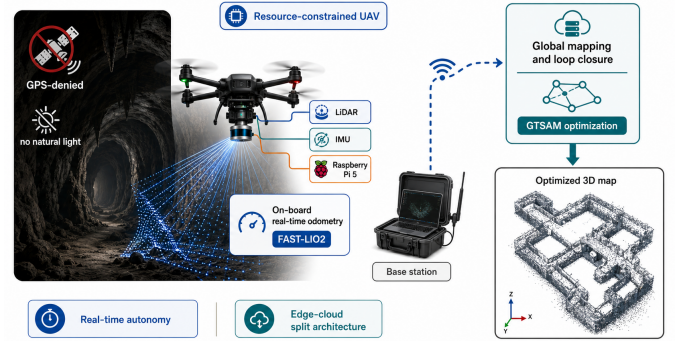


Fig. 1. Conceptual Overview of Edge-Assisted UAV Subterranean Mapping in GPS-Denied Environments

A. Core System Components

The system relies on three primary hardware components:

- **LiDAR:** A Hesai Pandar XT-16 mechanical rotating LiDAR provides 360-degree 3D point cloud coverage through 16 laser channels.
- **Inertial Measurement Unit:** An SBG Ellipse-D INS provides high-frequency (> 200 Hz) linear acceleration and angular velocity measurements.
- **Edge Compute:** A Raspberry Pi 5 (quad-core ARM Cortex-A76), chosen for its low cost and power consumption.

Accurate state estimation requires precise temporal alignment between sensors. Because mechanical LiDARs acquire points sequentially ($\Delta t = 0.1$ s for 10 Hz), continuous 6-DoF UAV motion during a single sweep causes severe point cloud distortion (skew). To compensate, a hardware Pulse-Per-Second (PPS) signal is routed directly from the SBG Ellipse-D to the Hesai XT-16. This microsecond-level synchronization locks their internal clocks, enabling the algorithm to perform mathematically accurate motion deskewing using continuous-time IMU pre-integration for each individual laser pulse.

TABLE I
COMPARISON OF CAVE EXPLORATION SOLUTIONS

Study	Technology	SLAM	Real-time	Limitations
Zhang et al. (SmartCaveDrone) [4]	RGB-D, Active Lighting	Yes	Yes	High power consumption, system complexity.
Papachristos et al. [2]	NIR, ToF	No	No	Poor performance in feature-sparse environments.
Weishampel et al. [5]	Airborne LiDAR	No	No	Limited to exterior cave entrance detection.
Grasso et al. (Bossea Cave) [6]	Handheld LiDAR, UAV	Yes	No (Offline)	Manual interior mapping, hours of post-processing required.
Tranzatto et al. (CERBERUS) [7]	Multi-modal UAVs, LiDAR	Yes	Yes	Extreme SWaP requirements severely limit UAV flight time (10-22 mins).
This Paper	LiDAR, IMU	Yes	Yes	Requires Performance Optimisation

B. Proposed Edge-Cloud Split Architecture

To bridge the gap between the severe hardware limitations of the UAV payload and the computational requirements of global SLAM, this system uses an edge-cloud split-compute architecture, as shown in Fig. 2. The edge processing layer runs exclusively on the Raspberry Pi 5 onboard the UAV. It manages sensor drivers, hardware time synchronization, and the FAST-LIO2 IESKF for high-frequency state estimation. The Pi 5 also performs keyframe extraction and generates Scan Context descriptors for loop closure detection. By restricting the Pi 5 to local odometry and descriptor generation instead of full factor graph optimisation, the system avoids excessive RAM usage and CPU overload during flight. The resulting odometry, downsampled point clouds, and loop closure constraints are saved locally and can optionally be transmitted through the ROS 2 telemetry link.

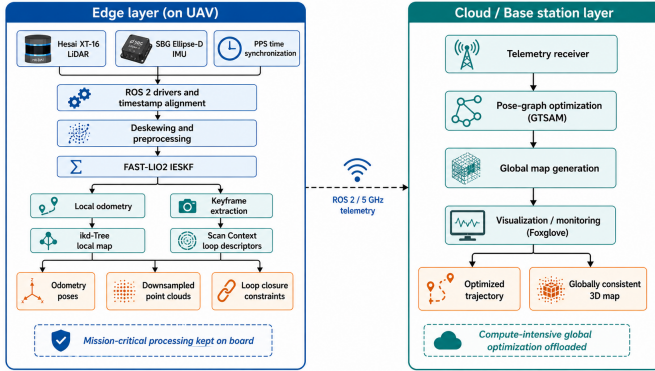


Fig. 2. Data flow and processing split between the UAV edge payload and the base station.

The cloud processing layer, executed on a high-performance base station or offline computer, is responsible for final global map construction. It receives the odometry poses and loop closure constraints generated by the Pi 5. A factor graph optimiser (GTSAM) [11] is then used to perform pose-graph optimisation, retrospectively correcting the trajectory drift accumulated during flight. This strict division of labor ensures the UAV maintains real-time navigational autonomy in the subterranean environment while offloading the heavy computational burden of global map optimisation.

C. State Estimation Formulation

To achieve real-time odometry on the Raspberry Pi 5, the system uses the FAST-LIO2 framework [10], which relies on an Iterated Error-State Kalman Filter (IESKF). The continuous-time kinematics are defined by an 18-dimensional state vector $\mathbf{x}$, which includes the UAV's global position $\mathbf{p}$, spatial rotation $\mathbf{R}$, velocity $\mathbf{v}$, IMU biases $\mathbf{b}_a$ and $\mathbf{b}_g$, and the local gravity vector $\mathbf{g}$:

$$\mathbf{x} = [\mathbf{p}^T \quad \mathbf{R}^T \quad \mathbf{v}^T \quad \mathbf{b}_a^T \quad \mathbf{b}_g^T \quad \mathbf{g}^T]^T \quad (1)$$

The main computational bottleneck in standard LiDAR-based Kalman filters is the calculation of the Kalman Gain ($\mathbf{K}$). In the traditional approach, the filter must invert the measurement covariance matrix:

$$\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R})^{-1} \quad (2)$$

where $\mathbf{R}$ is the measurement noise covariance matrix with dimensions $m \times m$, where m is the number of LiDAR points in the current scan. For dense subterranean point clouds, m often exceeds 10^4 points per sweep. Inverting this matrix requires $O(m^3)$ computational complexity, which quickly saturates the ARM CPU cores of embedded edge devices.

FAST-LIO2 addresses this hardware limitation mathematically by using the Matrix Inversion Lemma to restructure the Kalman Gain equation:

$$\mathbf{K} = (\mathbf{H}^T\mathbf{R}^{-1}\mathbf{H} + \mathbf{P}^{-1})^{-1}\mathbf{H}^T\mathbf{R}^{-1} \quad (3)$$

This transformation shifts the required matrix inversion from the measurement dimension ($m \times m$) to the fixed state dimension ($n = 18$). This reduces computational complexity from $O(m^3)$ to $O(n^3)$, enabling real-time processing on the Pi 5. Furthermore, map management uses an incremental kd-tree (ikd-Tree). By incrementally inserting N new points in $O(N \log M)$ time, rather than performing a complete $O(M \log M)$ static rebuild, the ikd-Tree significantly reduces peak RAM usage. This combination of $O(n^3)$ Kalman Gain computation and memory-efficient mapping provides the precise mathematical justification for selecting FAST-LIO2 over more computationally intensive factor-graph algorithms like LIO-SAM for this resource-constrained platform.

D. Global Loop Closure

While FAST-LIO2 provides high-frequency local odometry, it accumulates unbounded drift over extended subterranean trajectories. To address this, a global loop closure module based on Scan Context [12] runs locally on the Raspberry Pi 5. Scan Context encodes a 3D point cloud into a 2D global descriptor, making it highly efficient for the ARM processor. The point cloud is divided into azimuthal and radial bins. For each bin, the maximum height of the points within that bin is extracted to form a 2D matrix S . The value at row i and column j is defined as:

$$S(i, j) = \max_{p \in P_{i,j}} z(p) \quad (4)$$

where $P_{i,j}$ represents the set of points falling into the i -th radial ring and the j -th azimuthal sector, and $z(p)$ is the height of point p .

To achieve rotational invariance, Scan Context uses a column-shifting sequence. When comparing a candidate scan S^c with a query scan S^q , the distance d is computed by finding the minimum column-wise distance across all possible yaw shifts k :

$$d(S^q, S^c) = \min_{k \in [1, N_s]} \left(1 - \frac{\sum_{i,j} S^q(i, j) S_k^c(i, j)}{\sqrt{\sum_{i,j} (S^q(i, j))^2 \sum_{i,j} (S_k^c(i, j))^2}} \right) \quad (5)$$

where N_s is the total number of azimuthal sectors, and S_k^c is the candidate scan shifted by k columns. If the minimum distance d falls below a predefined threshold, a loop closure is registered. The Pi 5 continuously extracts these keyframes and computes the Scan Context descriptors during flight, saving the loop constraints for post-flight graph optimisation.

E. Software Architecture and Telemetry

The software pipeline is built on the Robot Operating System 2 (ROS 2) Humble middleware [13]. Its Data Distribution Service (DDS) architecture enables asynchronous, decentralised communication, preventing the Raspberry Pi 5 from being bottlenecked during high-frequency sensor ingestion. To manage high-bandwidth point cloud streams, LiDAR drivers are configured with a Best Effort Quality of Service (QoS) policy. This prioritises low-latency transmission over absolute completeness, ensuring that dropped UDP packets do not cause network queuing that would disrupt the continuous-time IESKF. Conversely, output odometry topics use a Reliable QoS policy to guarantee accurate trajectory tracking at the base station. To strictly preserve onboard computing resources for state estimation, the edge device operates completely headless. All graphical rendering and 3D visualization are offloaded to the remote base station via a 5 GHz Foxglove WebSocket connection [14]. This decoupled architecture isolates the Pi 5 from the significant processing overhead of rendering dense voxel grids, preventing runtime memory overflow and CPU saturation.

IV. EXPERIMENTAL SETUP

To validate the proposed edge-cloud split architecture and quantify the computational boundaries of embedded SLAM,

the system was evaluated using established dataset benchmarks processed on the edge hardware, then deployed on physical hardware.

A. Key Performance Indicators

System performance is evaluated against four Key Performance Indicators (KPIs):

- **Absolute Pose Error (APE):** Calculates the Root Mean Square Error (RMSE) of the global drift to validate trajectory accuracy. A low APE is critical for safe UAV return-to-home functionality.

$$\text{APE} = \sqrt{\frac{1}{N} \sum \|p_i - \hat{p}_i\|^2} \quad (6)$$

where p_i is the ground truth position, $\hat{p}_i$ is the estimated position, and N is the total number of estimated positions.

- **Processing Latency:** Ensures the system operates in real time. For autonomous flight, the latency between sensor acquisition and odometry output must remain below 100 ms (10 Hz), where t_{scan} is the acquisition timestamp of the LiDAR scan and t_{pub} is the publication timestamp of the resulting odometry.
- **CPU Load:** Acts as a safety margin to prevent system freezing. CPU load must not exceed 100% for extended periods, ensuring that output mapping frames are not dropped.
- **RAM Peak Usage:** Monitors memory allocation to prevent the operating system from swapping to the SD card, which causes fatal latency spikes or an Out-Of-Memory (OOM) hardware crash.

B. Dataset Evaluation

Initial benchmarking and edge-performance simulations were conducted directly on the Raspberry Pi 5 using the University of Michigan North Campus Long-Term (NCLT) Vision and LiDAR dataset [15]. Running the dataset on the physical Pi 5 accurately replicates the processor constraints and memory bottlenecks expected during live UAV operation, providing an empirical baseline for computational profiling before physical flight.

C. Physical Hardware Deployment

After dataset validation, the proposed edge-cloud architecture (Fig. 3a) was deployed on the physical sensor payload. A custom 3D-printed portable sensor mount (Fig. 3b) was fabricated to securely hold the Hesai XT-16 LiDAR, SBG Ellipse-D IMU, and Raspberry Pi 5. This rigid structure is essential for the FAST-LIO2 algorithm, as it maintains static translational and rotational extrinsic parameters between the sensors, which are necessary for accurate motion deskewing and state estimation. The live output of the mapping process is then transmitted over the 5 GHz telemetry link to the base station for real-time visualization.

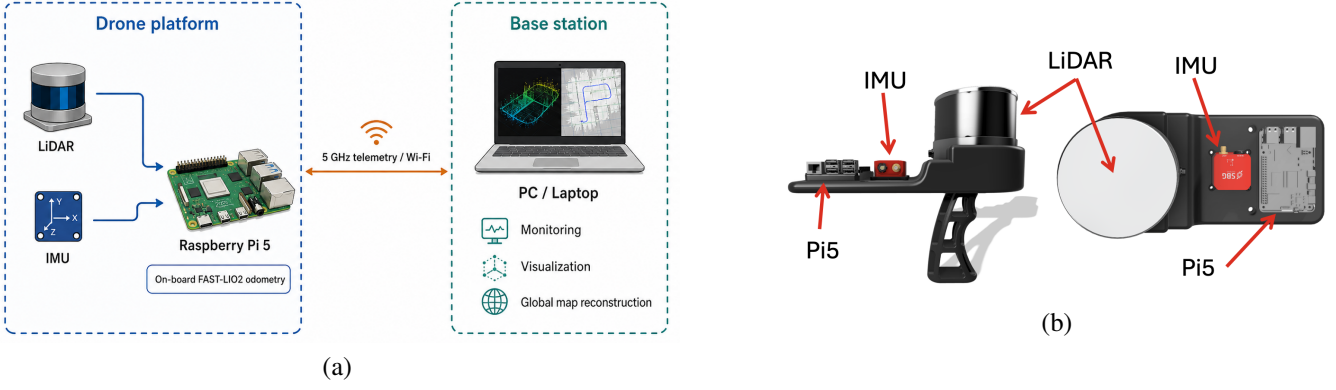


Fig. 3. The custom subterranean mapping payload. (a) Edge-Cloud System Architecture Overview. (b) CAD render of the custom 3D-printed sensor rig.

D. Experimental Methodology

The procedural details and independent variables for each experiment are described below.

- 1) **Odometry vs Global Mapping (RAM Saturation):** This experiment validates the necessity of the edge-cloud split-compute architecture by processing the NCLT dataset under two configurations: Odometry-Only and Global Mapping (which maintains a complete point cloud map in RAM). Memory allocation is logged to identify RAM usage that could lead to an Out-Of-Memory (OOM) hardware crash. The objective is to demonstrate that native global mapping on embedded hardware results in unsustainable RAM growth and CPU degradation, necessitating an odometry-only survival mode for flight and an edge cloud system for global mapping.
- 2) **Spatial Resolution vs. Compute Trade-off:** The FAST-LIO2 system uses a voxel grid filter to downsample raw LiDAR data into cubic volumes with side length v . According to the theoretical FAST-LIO2 latency model, processing time is expected to scale quadratically with the inverse of the voxel size:

$$T_{frame} \approx C_{hw} \cdot \left(\frac{P_{raw}}{k \cdot v^2} \right) \log(M) + T_{fixed} \quad (7)$$

A full parameter sweep across $k \in \{1, 2, 3, 4, 5\}$ and $v \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ m (25 configurations) was conducted on the Raspberry Pi 5 to empirically characterize this trade-off. The sweep was run on a physically collected outdoor dataset (a closed-loop circuit around the university campus lake, ~ 500 m), selected as the worst-case computational scenario since long-range unobstructed LiDAR returns populate a substantially larger spatial volume in the ikd-Tree map than an indoor environment. For each configuration, peak RAM usage, average per-frame processing latency, and frame drop rate were recorded. Configurations satisfying the real-time constraint (sub-100 ms latency, near-zero frame drop) were then re-evaluated using closing error as the Euclidean distance between the estimated start and end pose of the closed-loop trajectory. Since computational viability alone does not guarantee sufficient point density for reliable IESKF plane fitting, this

second sweep identifies the final operating configuration used in Experiment 3.

- 3) **Baseline Trajectory Error Analysis:** To assess the baseline accuracy and drift characteristics of the state estimation algorithm, the core FAST-LIO2 odometry is evaluated over a 1.146 km, 1,025-second trajectory from the NCLT dataset (2012-01-08 sequence) [15]. This analysis uses the optimal voxel grid settings determined from the results of Experiment 2. The accumulated error is measured by calculating the Absolute Pose Error (APE) between the real-time estimate and the high-precision RTK-GPS ground truth using the *evo* Python evaluation package [16].
- 4) **Physical Hardware Validation:** The custom payload was deployed across three physically distinct environments to validate the system under a range of operating conditions. The basement of the engineering building was used to simulate a GPS-denied, low-light subterranean environment under dark conditions, representing the closest physical analogue to the target cave deployment scenario. The exterior perimeter of the engineering building was used to evaluate performance in a structured outdoor environment with mixed line-of-sight geometry, while a closed-loop circuit around the campus lake was used to evaluate performance in an open, vegetation-heavy outdoor environment representing the most spatially demanding test case. A closed loop was walked in each environment to enable closing error measurement. Using the system configuration shown in Fig. 3, locally computed odometry is transmitted via the 5 GHz telemetry link for real-time visualization and monitoring on the base station laptop. The resulting global 3D maps are then overlaid on satellite imagery to verify hardware-software integration and global map consistency across all three environments.

V. RESULTS AND ANALYSIS

A. Edge vs. Global Mapping (RAM Saturation)

The memory allocation analysis over the 1,025-second dataset, shown in Fig. 5, empirically validates the need for the edge-cloud architecture. In the global mapping configuration, memory consumption increased linearly with processing time, peaking at 8.5 GB and resulting in an Out-Of-Memory (OOM) hardware crash. This continuous accumulation makes

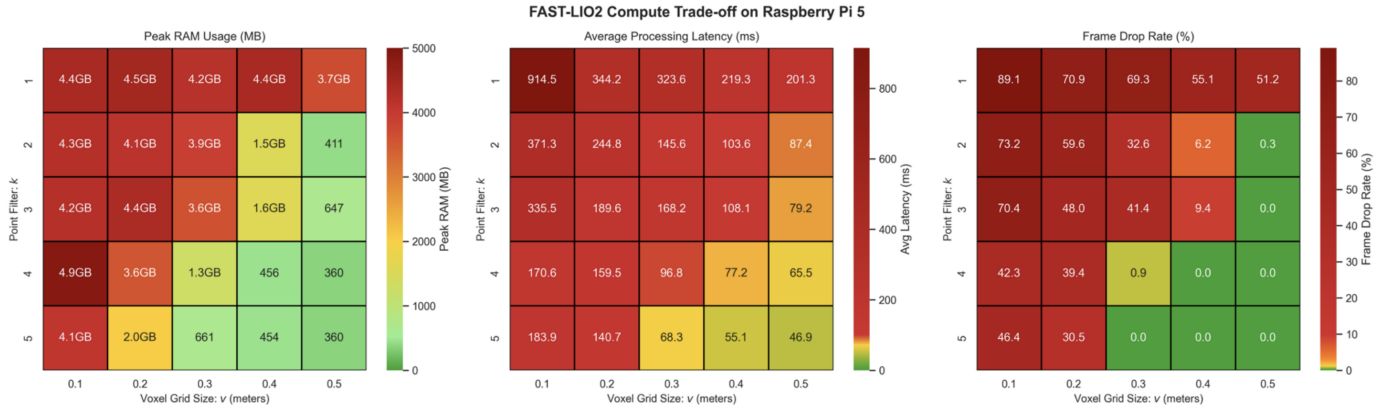


Fig. 4. FAST-LIO2 parameter sweep results: peak RAM usage (left), average processing latency (middle), frame drop rate (right)

native global mapping unsustainable for embedded hardware during extended missions. In contrast, the odometry-only configuration maintained a highly stable footprint, with peak memory allocation of just 588 MB. These results show that for long-duration autonomous flights on resource-constrained platforms, an odometry-only survival mode combined with a split-compute architecture is essential.

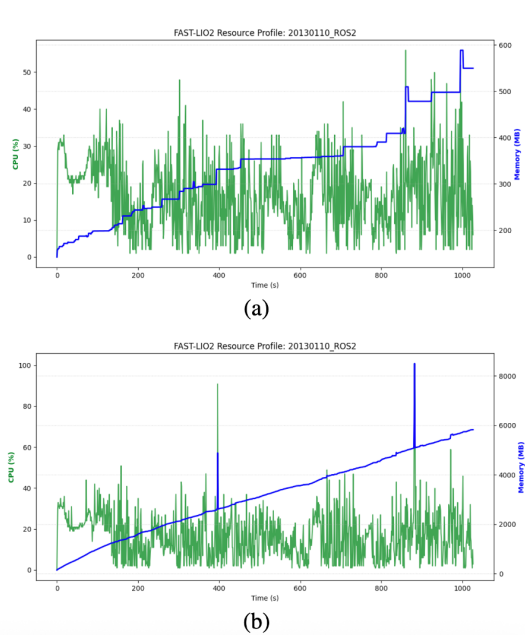


Fig. 5. Odometry vs Global Mapping (a) Odometry Mode (b) Global Mapping

B. Spatial Resolution vs. Compute Trade-off

The empirical results of the 25-configuration parameter sweep are shown in Fig. 4. The RAM heatmap shows a strong dependence on voxel size, with peak usage reaching 3.7–4.9 GB at $v = 0.1$ m across all k , falling to under 700 MB once $v \geq 0.3$ m. The latency heatmap follows the expected inverse relationship: the worst-case configuration ($k = 1$, $v = 0.1$) reaches 914.5 ms, over nine times the 100 ms real-time threshold, while the most aggressive configuration ($k = 5$, $v = 0.5$) achieves 46.9 ms. The frame drop heatmap defines the real-time boundary directly, with severe drop rates of 30–89% for $v \leq 0.2$ m regardless of k , falling to near-zero for $v \geq 0.3$ m combined with $k \geq 4$.

Computational feasibility alone, however, does not guarantee sufficient point density for reliable IESKF plane fitting. The viable configurations identified above were therefore re-evaluated using closing error as the accuracy metric, shown in Fig. 6. Configurations at $v \leq 0.2$ m diverge completely, with closing errors in the tens of kilometers, as the frame drops above break state propagation continuity. Accuracy improves sharply from $v = 0.3$ m onward, with the minimum closing error of 0.59 m achieved at $k = 5$, $v = 0.4$ m. This configuration is selected as the operating point for the remaining experiments, combining a 0.0% frame drop rate, 454 MB peak RAM, and 55.1 ms average latency, a 45 ms safety margin below the 10 Hz real-time threshold.

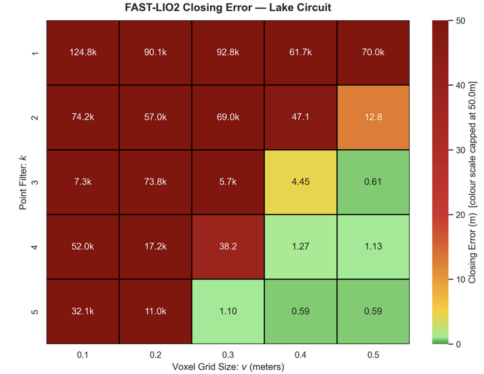


Fig. 6. FAST-LIO2 Closing Error Heatmap for Lake Circuit

C. Baseline Trajectory Error Analysis (APE)

The global consistency of the unoptimised edge odometry was evaluated against the RTK-GPS ground truth over the 1,146.31 m trajectory using SE(3) Umeyama alignment from the Evo Python package [16]. The results shown in Fig. 7 demonstrate exceptional state estimation stability and accuracy for a resource-constrained embedded platform. Running strictly in real time on the Raspberry Pi 5, the baseline FAST-LIO2 implementation achieved a Root Mean Square Error (RMSE) of 0.908 m and a mean APE of 0.834 m. Relative to the total flight distance, this corresponds to a highly precise mean translational error of 0.07%. As shown in the temporal plot in Fig. 7, the accumulated drift over the entire 1,025-second sequence peaked at a maximum deviation of only 2.228 m. This level of unoptimised accuracy provides robust

odometry for immediate collision avoidance and autonomous subterranean navigation. The minimal drift validates the capability of the edge computer, leaving only a minor 2 m global correction for the GTSAM pose-graph optimisation to resolve for the final map structure.

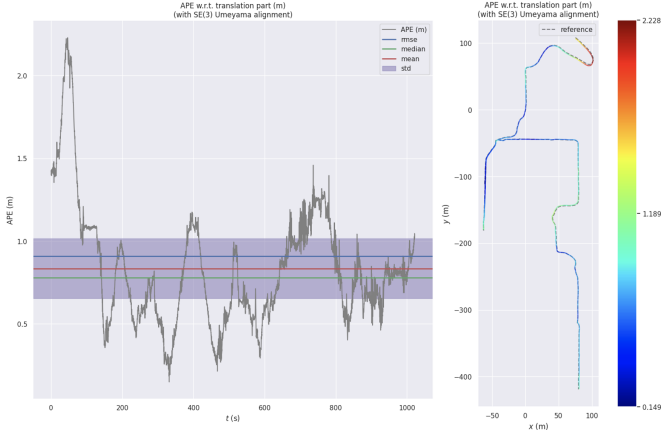


Fig. 7. FAST-LIO2 baseline trajectory error analysis on the NCLT 2013-01-10 sequence: APE over time (left), trajectory coloured by APE (right).

D. Physical Hardware Validation

The custom payload was deployed across all three environments. Throughout, the Raspberry Pi 5 maintained real-time survival odometry, executing the edge-cloud split without CPU saturation, memory overflow, or dropped frames. Table II summarises the results using the $k = 5$, $v = 0.4$ m configuration.

TABLE II
UCD DATASET RESULTS SUMMARY

Environment	Closing Err. (m)	Drift (%)	Latency (ms)	RAM (MB)	Drop (%)
Basement Corridor	0.53	0.25	19.5	178	<1
Lake Circuit	0.59	0.12	77.4	454	<1
Engineering Exterior	0.08	0.02	70.7	498	<1

All three deployments achieved sub-meter closing error with frame drop rates below 1%, confirming the configuration sustains real-time performance across both confined and open-outdoor geometries. The Basement Corridor result is the most significant, having been obtained under near-darkness, GPS-denied conditions in a featureless corridor, directly validating the system’s suitability for subterranean deployment.

The resulting point clouds were overlaid on satellite imagery for each environment, as shown in Fig. 8, with the alignment confirming the structural accuracy and global consistency of the edge-based SLAM architecture across all three deployments.

VI. CONCLUSION

This paper presented an edge-cloud split-compute architecture that addresses the severe Size, Weight, and Power (SWaP) constraints of autonomous subterranean UAVs. By offloading global map optimization, the onboard Raspberry Pi 5 executes real-time FAST-LIO2 survival odometry, avoiding the RAM saturation and hardware crashes inherent in native embedded

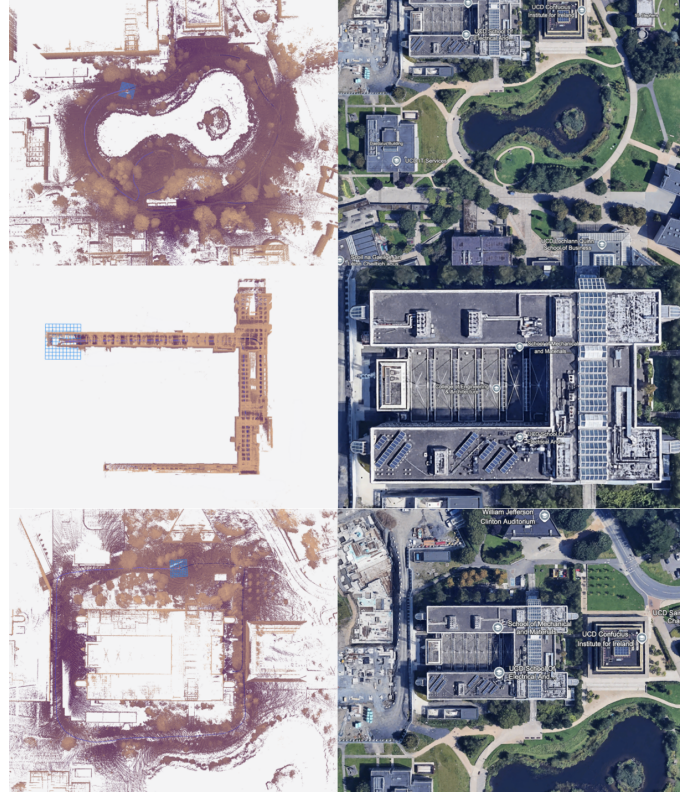


Fig. 8. FAST-LIO2 point cloud reconstructions (left) overlaid against satellite imagery (right) for the three UCD deployment environments. Top: Basement Corridor. Middle: Lake Circuit. Bottom: Engineering Building Exterior.

global mapping, reducing peak memory allocation by over 93% compared to native global mapping. A 25-configuration parameter sweep, combining computational feasibility with closing-error accuracy, identified $k = 5$, $v = 0.4$ m as the optimal operating point, sustaining a 0.0% frame drop rate with a 45 ms safety margin below the real-time threshold. Baseline accuracy was validated on the NCLT benchmark, achieving a 0.07% mean translational error, while physical deployment across three geometrically diverse environments, a GPS-denied basement corridor, an open lake circuit, and a structured building exterior, achieved sub-metre closing error in every case. The 0.53 m closing error obtained under near-darkness, GPS-denied conditions in the basement corridor directly validates the system’s suitability for subterranean deployment. To address severe telemetry attenuation in deep subterranean networks, future work will focus on integrating lightweight onboard path-planning and obstacle-avoidance modules, enabling fully autonomous return-to-home capabilities when base station connectivity is lost.

REFERENCES

- [1] GIM International. (2018) The tham luang cave rescue. [Online]. Available: <https://www.gim-international.com/>
- [2] C. Papachristos, S. Khattak, and K. Alexis, “Autonomous exploration of visually-degraded environments using aerial robots,” in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2017, pp. 775–780.
- [3] L. Huang, “Review on LiDAR-based SLAM techniques,” in *2021 International Conference on Signal Processing and Machine Learning (CONF-SPML)*. IEEE, 2021, pp. 163–168.

- [4] G. Zhang, B. Shang, Y. Chen, and H. Moyes, "SmartCaveDrone: 3d cave mapping using UAVs as robotic co-archaeologists," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 1052–1057.
- [5] J. F. Weishampel, J. N. Hightower, A. F. Chase, D. Z. Chase, and R. A. Patrick, "Detection and morphologic analysis of potential below-canopy cave openings in the karst landscape around the Maya polity of Caracol using airborne LiDAR," *Journal of Cave and Karst Studies*, vol. 73, no. 3, pp. 187–196, 2011.
- [6] N. Grasso, P. Dabove, and M. Piras, "The use of SLAM and UAV technology in geological field for monitoring: The case study of the Bossea Cave," *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLVIII-2/W3-2023, pp. 73–80, 2023.
- [7] M. Tranzatto *et al.*, "Team CERBERUS wins the DARPA subterranean challenge: Technical overview and lessons learned," *arXiv preprint arXiv:2207.04914*, 2022. [Online]. Available: <https://arxiv.org/abs/2207.04914>
- [8] J. Zhang and S. Singh, "LOAM: LiDAR odometry and mapping in real-time," in *Robotics: Science and Systems*, Berkeley, CA, 2014, pp. 1–9.
- [9] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled LiDAR inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [10] W. Xu *et al.*, "FAST-LIO2: Fast Direct LiDAR-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2021.
- [11] F. Dellaert, "Factor graphs and GTSAM: A hands-on introduction," Georgia Institute of Technology, Tech. Rep. GT-RIM-CP&R-2012-002, 2012.
- [12] G. Kim and A. Kim, "Scan Context: Egocentric Spatial Descriptor for Place Recognition within 3D Point Cloud Map," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Madrid, Spain: IEEE, 2018, pp. 4802–4809.
- [13] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [14] Foxglove, "Foxglove Studio: Robotics observability platform," 2026. [Online]. Available: <https://foxglove.dev/>
- [15] N. Carlevaris-Bianco, A. K. Ushani, and R. M. Eustice, "University of Michigan North Campus long-term vision and lidar dataset," *The International Journal of Robotics Research*, vol. 35, no. 9, pp. 1023–1035, 2016.
- [16] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM," 2017, gitHub repository. [Online]. Available: <https://github.com/MichaelGrupp/evo>

REFERENCES

- [1] GIM International, "The Tham Luang Cave Rescue," 2018. [Online]. Available: <https://www.gim-international.com/>
- [2] C. Papachristos, S. Khattak, and K. Alexis, "Autonomous exploration of visually-degraded environments using aerial robots," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2017, pp. 775–780.
- [3] L. Huang, "Review on lidar-based slam techniques," in *2021 International Conference on Signal Processing and Machine Learning (CONF-SPML)*. IEEE, 2021, pp. 163–168.
- [4] G. Zhang, B. Shang, Y. Chen, and H. Moyes, "Smartcavedrone: 3d cave mapping using uavs as robotic co-archaeologists," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 1052–1057.
- [5] J. F. Weishampel, J. N. Hightower, A. F. Chase, D. Z. Chase, and R. A. Patrick, "Detection and morphologic analysis of potential below-canopy cave openings in the karst landscape around the maya polity of caracol using airborne lidar," *Journal of Cave and Karst Studies*, vol. 73, no. 3, pp. 187–196, 2011.
- [6] N. Grasso, P. Dabove, and M. Piras, "The use of SLAM and UAV technology in geological field for monitoring: the case study of the Bossea Cave," *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLVIII-2/W3-2023, pp. 73–80, 2023.
- [7] M. Tranzatto *et al.*, "Team CERBERUS Wins the DARPA Subterranean Challenge: Technical Overview and Lessons Learned," *arXiv preprint arXiv:2207.04914*, 2022.